

Emacs Keybindings Cheat-Sheet

Doom Emacs, evil + evil-collection + rufwerk modules. Everyday keys first, occasional keys later.

Essentials

The three prefixes

SPC	leader, from normal/visual/motion state
M-SPC	leader, from insert or emacs state
SPC m	loclleader: commands for this major mode
M-SPC m	loclleader from insert state
SPC u	universal argument (replaces C-u)

Almost everything below hangs off SPC. If you forget a prefix, just press it and wait: which-key lists what follows.

When you are lost

SPC ?	context help for the current buffer
SPC h	Emacs help map (SPC h f function, SPC h v variable, SPC h k key)
SPC h b b	search every active keybinding
SPC :	M-x (also C-S-p)
SPC ;	eval an elisp expression
C-g	cancel the current command or prefix
ESC	leave insert state; quit a transient menu

Save, quit, undo

C-s	save buffer, from normal or insert state
SPC b s	save buffer SPC b S save all
u / C-r	undo / redo
SPC s u	browse undo history as a tree
SPC q q	quit Emacs SPC q r restart
SPC q s / SPC q l	quick-save / reload session

Copy and paste across the terminal boundary

C-S-c	copy region to the system clipboard
C-S-v	paste from the system clipboard
S-insert	paste (same thing, easier in a terminal)

Inside Emacs, plain y and p still work as usual; these three exist so that a terminal Emacs and the rest of the desktop share a clipboard.

Font and frame

C-= / C-	text scale up / down in this buffer
C-+	reset font size
M-C-= / M-C-	font size for the whole frame
C-S-f	toggle fullscreen
SPC t b	big font mode

Buffers and Windows

Switching buffers

SPC ,	switch buffer <i>within this workspace</i>
SPC <	switch buffer, all of them
SPC ~	back to the last buffer (the one key to learn)
SPC b [/ SPC b]	previous / next buffer
SPC b h	jump to the dashboard
SPC b x	scratch buffer in a popup SPC x toggles it

Killing buffers

SPC b k	kill this buffer (SPC b d is the same)
SPC b 0	kill every <i>other</i> buffer
SPC b Z	kill buried buffers
SPC b K	kill all buffers

Windows SPC w

SPC w v / SPC w s	split vertical / horizontal
SPC w V / SPC w S	split and move into the new window
SPC w c	close this window SPC w n new one
SPC w m m	maximize: close all the others
SPC w o	enlargen this one, keeping the rest
SPC w h j k l	move focus left, down, up, right
SPC w H J K L	move the window itself
SPC w =	rebalance SPC w x swap two windows
SPC w u / SPC w C-r	undo / redo the last window change
SPC w w	cycle SPC w C-w pick one visually

Doom puts **enlargen** on SPC w o, where vanilla evil closes the other windows. SPC w m m is the one that closes them.

Spatial motion (no leader needed)

M-+ ↓ ↑ →	move focus to the panel in that direction
C-M-+ ↓ ↑ →	same, for terminals that eat plain M-
SPC w + ↓ ↑ →	same, via the leader

Saved layouts

SPC w 1	code + terminal
SPC w 2	code + Machine Spirit
SPC w 3	writing
SPC o .	workflow transient: layouts, terminals, sessions

Popups

SPC ~	toggle the last popup
q	close the popup you are in

Workspaces, Projects, Files

Workspaces SPC TAB

Each project gets its own workspace, with its own buffer list and its own drawer terminal.

SPC TAB TAB	show the tab bar
SPC TAB .	switch by name
SPC TAB ~	last workspace
SPC TAB 1 ... 9	switch by position (also M-1 ... M-9)
SPC TAB] / [	next / previous
SPC TAB n / N	new / new and named (also C-t)
SPC TAB r	rename
SPC TAB d	kill this workspace
SPC TAB k	kill a workspace you pick
SPC TAB K	kill all the others
SPC TAB R	restore the last session
SPC TAB x	kill the whole session (asks first)

Projects SPC p

SPC p p	switch project, opening it in a fresh workspace
SPC p D	discover projects under the search roots
SPC p .	project dashboard
SPC p b	switch to a buffer in this project
SPC p a	add a project by hand
SPC p i	invalidate the file cache when new files go missing
SPC p k	kill this project's buffers
SPC p t / SPC p T	run a task / rerun the last one
SPC p c	compile SPC p ! shell command at the root

Files SPC f

SPC SPC	find file in project (also C-p)
SPC .	find file from here
SPC f r	recent files
SPC f f	find file SPC f F relative to this buffer
SPC f s / SPC f S	save / save as
SPC f R	rename or move this file
SPC f D	delete this file
SPC f y / SPC f Y	yank its path / its project-relative path
SPC f p / SPC f P	find in / browse the Doom config
SPC f u	reopen this file as root

Directories

SPC o -	diried on this file's directory
SPC o p	toggle the Treemacs sidebar
SPC o /	dirvish over a directory you pick

Editing Quickly

The grammar

An edit is **operator + motion or text object**. Learn the pieces once and every combination works.

d c y v	delete, change, yank, select
g U / g u	upcase / downcase
g q	re-fill (wrap) g Q format region
g c	comment (g c c line, g c i p paragraph)
=	indent

Text objects: i inner, a around

w / W	word / WORD
p	paragraph s sentence
([{ " ' ,	the matching pair
t	tag (HTML/JSEX)
f	function c comment
a	argument i indent block
q	whatever quotes this u URL
B	the nearest block g the whole buffer

So c i " changes what is inside the quotes, d a f deletes the whole function, y i p yanks the paragraph.

Surround

y s obj char	surround an object
d s char	delete the surrounding pair
c s old new	replace one pair with another
S char	in visual state, wrap the selection

So y s i w " quotes the word, c s " ' swaps the quotes, d s (strips the parentheses.

Grow the selection

SPC v	expand region: word → string → block → function
SPC V	contract it again

Many cursors at once g z

g z d / g z D	cursor here, jump to next / previous match
g z m	a cursor on every match in the buffer
g z j / g z k	drop a cursor on the line below / above
g z s	skip this match and move on
g z u / g z q	undo the last cursor / all of them
I / A	in visual block: insert at the start / end of every line

Or edit matches in place: M-d adds the next occurrence of the symbol, v R multiedits every match in the selection.

Snippets and folds

TAB	expand the snippet before point; jump to the next field
SPC i s	insert a snippet from the list
SPC i y	paste from the kill ring
SPC i u	insert a unicode character by name
z a z c z o	toggle / close / open the fold at point
z m / z r	fold everything / unfold everything

Jumping inside the buffer

g g / G	top / bottom { } by paragraph
%	jump to the matching bracket
C-o / C-i	back / forward through the jump list
^^	return to where you were before the last jump
f / t	jump to a character on this line (; repeats)
g s SPC	jump to any visible position by typing it (avy)
g s s	jump by two characters
g s	is the easymotion prefix, not surround. Surround is y s, c s,
d s	above.

Searching and Jumping

Search SPC s

SPC /	search the project with ripgrep (SPC s p)
SPC *	search the project for the symbol at point
SPC s s	search this buffer SPC s S for the thing at point
SPC s B	search every open buffer
SPC s d / SPC s D	search this directory / another one
SPC s P	search a project other than this one
SPC s i	jump to a symbol in this buffer (imenu)
SPC s j / SPC s r	jump list / marks
SPC s l	jump to a visible link
SPC '	resume the last search where it left off

Inside a search or completion prompt

C-j / C-k	next / previous candidate
C-S-j / C-S-k	scroll the candidate list
TAB	complete as far as the candidates agree
C-;	act on the candidate (embark)
C-c C-e	export the results to an editable buffer
C-c C-;	export read-only C-c C-l collect
ESC	abort

The exported buffer is the trick worth remembering: run a project search, export it, edit the matches, and the edits write back to every file at once (wgrep).

Jump to code

g d	definition g D references
K	documentation for the thing at point
g f	open the file named at point
SPC c j	jump to a symbol anywhere in the project
SPC c i	implementations SPC c t type definition
C-o	come back

Bookmarks and marks

SPC RET	jump to a bookmark
SPC b m / SPC b M	set / delete a bookmark
mæ `æ	set mark x / jump to it

Bracket motions:] forward, [back

] e / [e	error
] d / [d	changed hunk since the last commit
] t / [t	TODO / FIXME comment
] m / [m	start of a method] M its end
] c / [c	comment block
] a / [a	argument
] f / [f	file in this directory
] w / [w	workspace
] o / [o	open a blank line below / above

Look things up

SPC s o	look up the selection online
SPC s t / SPC s T	dictionary / thesaurus
SPC s k	search the local docsets

Writing Code SPC c

LSP (eglot)

SPC c a	code actions at point: the fix-it menu
SPC c r	rename this symbol everywhere
SPC c d / SPC c D	definition / references
SPC c k	documentation SPC c i implementations
SPC c j	jump to a project symbol
SPC c x	list diagnostics for the buffer
] e / [e	next / previous error

Format and clean

SPC c f	format the region, or the whole buffer
g Q	format just the motion you give it
SPC c w / SPC c W	strip trailing whitespace / newlines

Formatting also runs on save (+onsave), so SPC c f is for when you want it before then.

Run it

SPC c e	evaluate the buffer or region
M-r	evaluate the buffer, no leader
SPC c E	evaluate the region and replace it with the result
SPC c s	send the buffer or region to a REPL
SPC o r	open a REPL in another window
SPC c c / SPC c C	compile / recompile

Tests

SPC p v	test the current file
SPC p V	test the package or project
SPC p C-v	rerun the last test invocation

Debugger SPC d

(dape)	
SPC d d	start a session SPC d c continue
SPC d b	toggle a breakpoint SPC d B remove all
SPC d n s o	step over / into / out
SPC d x	evaluate an expression in the stopped frame
SPC d i / SPC d R	info panes / debug REPL
SPC d q	quit the session

Syntax and spelling

SPC t f	toggle the syntax checker
z =	correct the misspelling at point (jinx)
Z =	switch spellcheck languages

Magit: The Daily Loop

Evil owns the lowercase letters in magit buffers, so several keys differ from the vanilla docs online. Where they differ, the vanilla key is noted.

Open, refresh, quit

SPC g g	magit status SPC g G status at this file
h	dispatch: the menu of every menu (also ?)
g r / g R	refresh this buffer / everything
q / Q	bury this buffer / all magit buffers
`	process buffer: <i>this is where git's error text is</i>
	run an arbitrary git command

Moving around the status buffer

j / k	line down / up, as everywhere else
C-j / C-k	next / previous section
] / [	next / previous <i>sibling</i> section
g h	up to the parent section
TAB or z a	fold or unfold the section at point
z 1 . . . z 4	refold the whole buffer to that depth
g u / g s	jump to unstaged / staged
g t / g n	jump to tracked / untracked
S-SPC / S-DEL	peek at the diff and scroll it
RET	visit the thing at point

Stage and commit

s / u	stage / unstage the thing at point
S / U	stage everything modified / unstage everything
x	discard the change at point (<i>vanilla k</i>)
c c	commit
c a	amend c e extend, keeping the message
c w	reword the message c f fixup
C-c C-c	finish the message and commit
C-c C-k	abort the message

Staging is recursive: put point on a hunk and s stages the hunk, on a file and it stages the file, on the section header and it stages the lot. Select a few lines in visual state first to stage exactly those.

Sync with the remote

f a	fetch all remotes f p fetch the pushremote
f n	fetch forge topics (PRs, issues) f N notifications
F p / F u	pull from pushremote / upstream
F n	same forge-topic fetch, from the pull menu
p p / p u	push to pushremote / upstream
p -f	toggle force, then push <i>use with care</i>

Push is lowercase p here. Vanilla magit uses P, and p moves between sections; evil-collection swaps them so k and p stay vim-ish. Forge's PR list is separate from git fetch: use f n (or @ f f) before the Pull Requests section / SPC g l p can show anything new.

Review a PR (forge)

RET	on a PR in status: open its topic buffer
C-c C-n	new comment C-c C-c submit C-c C-k abort
C-c RET	topic menu: act on the PR at point
/ A / / R	in that menu: approve / request changes

Approve opens a post buffer for the review text (fine to leave empty); C-c C-c submits it to GitHub *immediately* — forge has no

draft-review state.

Git without opening magit

SPC g s / SPC g r	stage / revert the hunk under point
] d / [d	next / previous changed hunk
SPC g S / SPC g U	stage / unstage this whole file
SPC g B	blame this file
SPC g L	log for this file only
SPC g t	step through the file's history (time machine)
SPC g y	copy a link to this line on the remote
SPC g /	magit dispatch SPC g . file dispatch

Magit: Branches and Merges

Branches b

b b	checkout a branch or revision
b l	checkout a local branch
b c	create a branch and check it out
b n	create a branch and stay where you are
b s	spin off: take your recent commits with you
b m	rename b x reset to another ref
b k	delete a branch (<i>vanilla b x</i>)

Stash z (capital)

Z z	stash the index and the worktree
Z i / Z w	stash only the index / only the worktree
Z a / Z p	apply / pop the stash at point
Z l / Z v	list / show
Z b	start a branch from a stash
Z Z	snapshot: stash but keep your changes in place
Z k	drop a stash

Lowercase z is folding here, so stash moved up to Z.

Merge m

m m	merge, committing if it applies cleanly
m e	merge and edit the message first
m n	merge but do not commit
m s	squash merge
m p	preview what a merge would do
m a	absorb: merge the branch and delete it

While a merge is in progress the same prefix changes meaning: m m commits the merge and m a *aborts* it. “No commit” only affects the message; a conflict always stops the merge regardless.

Rebase	r
r i	interactive rebase from a commit you pick
r u / r e	rebase onto upstream / onto anything
r s	interactive rebase of a subset
r r / r a	continue / abort a rebase in progress
r f	autosquash the fixups

In the todo buffer: **p** pick, **r** reword, **e** edit, **s** squash, **f** fixup, **d** drop, **g j** / **g k** move a line down / up, then **C-c C-c** to start.

Conflicts	
e	on a conflicted file: open a 3-way ediff
RET	open the conflicted file normally

In ediff: **n** / **p** between conflicts, **a** take ours, **b** take theirs, **q** quit.

In the file itself (smerge): **C-c ^ n** next conflict, **C-c ^ u** keep upper (ours), **C-c ^ l** keep lower (theirs), **C-c ^ a** keep both. Then stage with **s** and finish with **m m**.

Magit: History, Worktrees, Forge

Log and diff

l l	log the current branch	l o another one
l b	all branches	l a all references
l r	reflog: what HEAD has actually pointed at	
d d	diff dwim	d r diff a range
d u / d s	diff unstaged / staged	
d w	diff the worktree against a ref	
= / +	show less / more context in a diff	
g =	back to the default context	
y r	show refs	y s copy the value at point
y b	copy this buffer's revision	

Rewriting and rescuing

A	cherry-pick the commit at point
a	apply a cherry without committing
- / _	revert without committing / revert
o / O	reset quickly / the reset menu (<i>vanilla</i> x X)
W	create or apply a patch w apply a mailbox
B	bisect
!	run a git command or script

If a reset or rebase loses something, the reflog (**l r**) still has the old head. Put point on it and reset back with **o**.

Worktrees **% (or *)**

% b	checkout an existing branch into a new worktree
% c	create a branch and a worktree for it
% g	visit another worktree's status buffer
% m	move a worktree
% k	delete one

Picking **origin/foo** creates the local **foo** for you, so you do not end up detached. Put worktrees under **~/Projects/** so projectile finds them.

Branch-to-workspace flow

% b	make the worktree
SPC p D	let projectile discover it
SPC p p	open it in its own workspace

Each worktree then has its own buffers, its own terminal, and its own agent session.

Forge (GitHub)		@
@	forge dispatch	SPC g ' from anywhere
@ f f	fetch topics from GitHub (PRs, issues)	
@ f n	fetch notifications only	
f n / F n	same fetch, from Magit's fetch / pull menus	
@ c i / @ c p	create an issue / a pull request	
g i / g m	jump to the issues / pull requests section	
RET	on a topic: open it	
C-c C-n	start a comment	C-c C-c submit
C-c C-k	throw the comment away	

SPC g l p /	list cached pull requests / issues
SPC g l i	
SPC g f p /	visit a PR / an issue by number
SPC g f i	
SPC g o p /	open one in the browser
SPC g o i	

Fetch first with **@ f f** or **f n**; the list commands only show what forge has already pulled into its local database.

Terminals

The drawer

One terminal per workspace, opening at the bottom of the frame.

C-`	toggle the drawer, from anywhere
SPC o t	the same, via the leader
SPC o T	open a new named terminal
SPC o TAB	switch to (or adopt) another terminal
SPC o] / SPC o [	next / previous terminal

Eat's three modes

The terminal has to decide who gets your keystrokes.

C-c C-j	semi-char: typing goes to the shell (the default)
C-c C-e	emacs mode: the buffer is read-only, Emacs keys work
C-c M-d	char mode: <i>everything</i> goes to the shell
C-c C-l	line mode: edit a line, then send it

Use emacs mode when you want to search or copy out of the scrollback, char mode when a TUI needs a key that Emacs would otherwise steal.

Inside the terminal

C-c C-z	toggle whether ESC reaches the shell
C-c C-p / C-c C-n	jump to the previous / next shell prompt
C-c C-k	kill the process running in this terminal
C-S-v or S-insert	paste
ESC then SPC b k	close the terminal buffer itself

C-c C-z matters for TUIs like vim or a full-screen pager: by default Emacs swallows **ESC**, and the advice in **config.el** is what makes this toggle work at all under evil.

Elsewhere

SPC p !	one-off shell command at the project root
SPC p &	the same, asynchronously

For anything longer-lived, **SPC :** then **async-shell-command**.

AI Assistants

Agent shell (ACP) **SPC l a**

SPC l a	this workspace's agent shell
SPC u SPC l a	force a fresh session here
SPC l r	resume an earlier session
RET	submit the prompt
ESC then SPC b k	kill the agent buffer

One session per workspace; the header line shows its directory and branch. Reads and edits are auto-approved, shell commands and deletions still ask. “Busy, please wait” means the session is still starting, or its handshake died and it needs killing and relaunching.

Machine Spirit **SPC l**

SPC 1 E		open or toggle the panel
SPC 1 N	S K	new / switch / close session
SPC 1 L		list sessions
SPC 1 M		magit status for one
SPC 1 v		review the changes it made
SPC 1 T		timeline of what it did
SPC 1 A		accept everything
SPC 1 B / SPC 1 R		checkpoint / revert to one
SPC 1 C		clean up stale worktrees
SPC 1 d		dictate a prompt
C-c C-b		checkpoint from inside the panel

Machine Spirit renders subagent-heavy work better than the agent shell does; reach for it when a skill fans out into many sub-tasks.

Dictation and inline LLM

SPC i d	dictate text at point (Whisper)
SPC o l l	open gptel SPC o l s send
SPC o l e	explain the selection
SPC o l r	rewrite the selection
SPC o l a /	add the region / a file to the context
SPC o l f	

Which one to use

quick question	gptel, SPC o l e
edit this repo	agent shell, SPC l a
multi-step work	Machine Spirit, SPC l E

Org and Notes	SPC n
---------------	-------

Capture and agenda

SPC X	capture a note right now
SPC n n / SPC n N	capture / go to the capture target
SPC n a	agenda SPC n t the todo list
SPC o h	agenda, habits view
SPC o g	time blocks
SPC n c / SPC n C	toggle the last clock / cancel it
SPC n o	jump to the running clock
SPC n l	store a link to here

Inside an org buffer

TAB / S-TAB	fold this heading / the whole outline
M-RET	new heading at the same level
M-+ →	promote / demote
M-↑ ↓	move the subtree
SPC m t	cycle the TODO state SPC m q set tags
SPC m d d	deadline SPC m d s schedule
SPC m l l	insert a link SPC m l s store one
SPC m x	toggle a checkbox SPC m ’ edit the block
SPC m e	export dispatcher
C-c C-c	do the obvious thing here (tags, tables, blocks)

Finding notes

SPC n s	search the notes and the vault together
SPC n f / SPC n F	find / browse a note file
SPC n g	clip the region into the notes
SPC n h	note history for this file
SPC n m	tag search SPC n v full-text view

Obsidian vault

SPC n w	
SPC n w j	jump to a note
SPC n w c	capture SPC n w s search the vault
SPC n w t	find by tag
SPC n w l	insert a wikilink SPC n w f follow one
SPC n w b	toggle the backlinks panel
SPC n w g /	whole graph / local graph
SPC n w G	
SPC n w o	open this note in Obsidian itself
SPC n w r	rescan the cache when notes look stale

PDFs and annotation

SPC n p	start or resume org-noter on this PDF
= / -	zoom in / out 0 or W fit width
P	fit the page SPC m m midnight mode

Writing Prose

Get out of the way

SPC t z	olivetti: center the text column
SPC m R	read mode, in org and markdown
SPC t w	soft line wrapping
SPC t l	line numbers off (or on)
SPC t r	read-only

Spelling and shape

z =	correct the word at point
Z =	change the spellcheck language
g q i p	re-wrap this paragraph
g w	re-wrap without moving point

Markdown

SPC m P	render this file to PDF
SPC m R	read mode
SPC m e f	Substack preflight check
SPC m e h	draft history for this file

Publishing to Substack SPC m e

SPC m e s	push as a draft
SPC m e p	publish
SPC m e n	post as a note
SPC m e S	export to the clipboard instead
SPC n w n	start a new post in the vault
SPC o s	the Substack action menu
SPC o S	sync the list of accounts you follow

Run the preflight (SPC m e f) before pushing: it is what catches the markup Substack silently mangles.

LaTeX documents

SPC m s	side-by-side source and PDF
SPC m c	compile SPC m a run the whole chain
SPC m v	jump to this point in the PDF
SPC m m	pick an AUCTeX command by hand
SPC m p SPC m P	preview / unpreview at point
C-c C-c	run the next sensible command

Math and Equations	SPC n e
--------------------	---------

Notebook and scratch

SPC n e n	open the org math notebook
SPC n e s	math scratchpad
SPC n e p	preview the equation at point

Insert

SPC n e e	a numbered equation
SPC n e a	an aligned derivation
SPC n e m	a matrix (asks rows, columns, environment)
SPC n e l	an equation label SPC n e r a reference

Snippets that fire inside math

Typed inside \ (... \) or \$... \$, these expand as you type (laas).	
;E ;V ;C ;P	E, Var, Cov, Pr
;a ;b ;g	Greek letters
;;d	∂ ;8 ∞
x1	x1 x/ a fraction
sr / cb	squared / cubed
argmin argmax	operators with a subscript slot
plim ols	plim, βOLS

Equations render to images when point leaves them (org-fragtog). Move back into one to edit the source again.

Check the algebra

SPC n e c	evaluate with Calc
SPC n e C	embedded Calc, in place
SPC n e y	verify with SymPy
SPC n e x	ask the model to explain this equation
SPC n e t	ask it to convert the equation

Tablet round trip

SPC n e i	import handwritten math from the reMarkable
SPC n e o	review the OCR transcription
SPC n e R	send the write-up to the tablet

From notes to a write-up

SPC n e w	promote the notes into a write-up
SPC n e P	export the write-up as PDF

Research TasksSPC p r

The **tasks**/ pipeline: **download_*** → **clean_*** → **merge_*** → analysis → **plot_*/tabulate_***. Tasks read upstream output only through **../input** symlinks declared in their Makefiles.

Navigate a task

SPC p r n	scaffold a new task
SPC p r r	this task's readme
SPC p r m	its Makefile
SPC p r o	its output directory, in dired
SPC p r u /	jump upstream / downstream
SPC p r d	
SPC p r g	the task-flow graph (also SPC p G)

Run things

SPC p r t	run this task with make (also SPC p m)
SPC p t / SPC p T	run / rerun a project task
SPC p c	compile the project

Logbook and slides

SPC p r l	new logbook entry
SPC p r L	build the logbook PDF
SPC p r s	build the slide deck

Data files

SPC m a / SPC m u	align / unalign CSV fields
SPC m s / SPC m S	sort by field / sort numerically
SPC m k	kill a field SPC m t transpose

Statistics languages

SPC c s	send the buffer or region to the REPL
SPC o r	open a REPL beside the code
SPC m	the localleader menu for R, Julia, Stata

Web DevSPC p w

Dev server and preview

SPC p w d	start the dev server and open the preview
SPC p w o	open the live preview
SPC p w k	stop this dev server
SPC p w r	run an npm script
SPC p w i	inspect the live React page
SPC p w x	kill every dev server and container

Parallel worktrees each get their own server: the first runs the full dev, the rest run vite only and bump their ports automatically.

Storybook

SPC p w s	start Storybook SPC p w K stop it
SPC p w b	open a story
SPC p w f	jump to a story's source
SPC p w e	explain a component

Scrutiny

SPC p s	the Scrutiny command center
SPC p v / SPC p V	test this file / the project
SPC p C-v	rerun the last test
SPC p w a	rebuild the agent containers
SPC o D	docker

Poking an API

C-c C-c	in a .http buffer: send the request at point
C-c C-r	send it and show the raw response
C-c C-v	send it but stay where you are

Study ModeSPC n S

SPC n S → pick a subject → the world tree of curriculum progress → enter a world. You get a submission buffer with the tutor's transcript beside it. The tutor grades against a stored solution without showing it to you.

In the submission buffer

C-c C-c	submit the attempt for grading
C-c C-h	the hint ladder
C-c C-n	next problem C-c C-e an easier one
C-c C-a	ask the tutor a free-form question
C-c C-m	materials for this problem
C-c C-s	the field's cheat sheet
C-c ?	how to type math here
C-c C-t	switch between flashcards and long form
C-c C-d	dictate the argument instead of typing it
C-c C-g	record the result by hand
C-c C-q	restart the tutor if it wedges

The hint ladderC-c C-h

Escalate only after a real attempt.

1	redirect: a concept or a reference to look at
2	structure: the skeleton of the argument
3	one step worked, then handed back to you

Without spending a hint:

c	am I on track? (no grading)
a	ask a question
m	materials e deal an easier problem

The world tree

RET	enter the world at point
m	its materials c the cheat sheet
l	lectures u unpack more problems
x	sit an exam W review your weak spots
g	refresh q quit

In the field list: RET enter, N add a field, l lectures, g refresh.

Handwriting on the tablet

C-c C-r	send this problem to the reMarkable
C-c C-i	import the handwritten attempt back

Reviewing what the tutor wrote

C-c C-c	accept the suggested edit
C-c C-k	discard it

Mail, Feeds, Calendar

MailSPC o m

SPC o m	open mu4e
C	compose R reply F forward
W	reply to everyone
C-c C-c	send C-c C-k discard the draft
s	search / narrow the current search
J	jump to a maildir b a bookmark
RET	read [[]] previous / next unread

Marks are set first and applied afterwards:

d	trash D delete outright
r	refile m move somewhere you pick
!	read ? unread + flag
u	unmark this U unmark everything
x	execute every mark you set

SPC o L finishes a pending DavMail login.

FeedsSPC o r

SPC o r	open elfeed
SPC o R	set the filter from anywhere
g	update the feeds
f	change the filter (your binding)
RET	preview the entry in the side pane
b	open it properly (browser or reader)
u	mark read U mark unread again
s	live filter c clear it
+ -	add / remove a tag
y	yank the entry's URL
=	the scoring rule keymap

Entries are ranked by `elfeed.score`; anything below the mark threshold is auto-read, and unread items older than two weeks expire, so `+unread` really means recent.

YouTube and Substack

<code>SPC o y</code>	the YouTube subscription menu
<code>SPC o y a / d</code>	subscribe / unsubscribe
<code>SPC o y l / y</code>	list subscriptions / open the video feed
<code>SPC o s</code>	Substack actions
<code>SPC o S</code>	sync the accounts you follow

Calendar

<code>SPC o c</code>	the calendar view
<code>SPC o C</code>	sync every calendar
<code>SPC o g</code>	time blocks
<code>SPC o h</code>	the habits agenda

Browsing

<code>SPC o w</code>	open a URL in EWW, inside Emacs
<code>SPC o W</code>	open it in the system browser
<code>SPC s l</code>	jump to a visible link on the page

Config and Recovery

Editing the config

<code>SPC f p</code>	find a file in <code>~/ .config/ doom</code>
<code>SPC f P</code>	browse the config directory
<code>SPC h r r</code>	reload the config in place
<code>SPC c e</code>	eval the elisp you just changed

Modules load in numeric order from `config.el`, and `local.el` loads first so a plain `setq` there beats every module default.

After changing `init.el`

<code>doom sync</code>	install or remove packages
<code>doom sync -u</code>	and update them
<code>doom doctor</code>	diagnose a broken setup
<code>doom build</code>	recompile after an Emacs upgrade

Health checks from inside Emacs

<code>SPC h D</code>	the rufwerk doctor: is every external tool present?
<code>SPC h T</code>	the config test suite (<code>make test</code>)
<code>SPC h B</code>	re-run bootstrap
<code>SPC t t</code>	set the current theme permanently

When something misbehaves

<code>SPC h k</code>	what is this key actually bound to?
<code>SPC h b b</code>	search every binding
<code>SPC h f / SPC h v</code>	describe a function / a variable
<code>SPC h e</code>	the <code>*Messages*</code> buffer

For a backtrace on the next failure: `SPC :` then `toggle-debug-on-error`.

Sessions

<code>SPC q s / SPC q l</code>	quick-save / reload the session
<code>SPC q S / SPC q L</code>	save to / load from a named file
<code>SPC q r</code>	restart Emacs
<code>SPC q d</code>	restart the daemon
<code>SPC q q / SPC q K</code>	quit / <code>quit and kill the daemon</code>

Insert-state ESC

<code>M-<ESC></code>	toggle whether <code>ESC</code> leaves insert state
<code>SPC t e</code>	the same, via the leader

Turn it off when a paste or a TUI keeps kicking you into normal state.

Gotchas

Capitals matter

<code>Z k</code>	drops a <code>stash</code>
<code>b k</code>	deletes a <code>branch</code>
<code>% k</code>	deletes a <code>worktree</code>
<code>x</code>	discards the change at point, no undo
<code>o / O</code>	resets (vanilla magit puts these on <code>x X</code>)
<code>SPC TAB x</code>	kills every workspace (it asks first)
<code>SPC p w x</code>	kills every dev server and container

Where the online docs will lie to you

Evil owns the lowercase letters in magit, so vanilla tutorials disagree:

<code>p</code>	push (<i>vanilla P</i>)
<code>x</code>	discard (<i>vanilla k</i>)
<code>o / O</code>	reset (<i>vanilla x X</i>)
<code>Z</code>	stash (<i>vanilla z; z is folding now</i>)
<code>% or *</code>	worktree (<i>vanilla Z</i>)
<code>- / -</code>	revert (<i>vanilla v V</i>)
<code>`</code>	process buffer (<i>vanilla \$</i>)
<code>y r y s y b</code>	show refs, copy value, copy revision

Keys this config moved

<code>g z</code>	refresh, in magit status. It is <i>not</i> jump-to-stashes any more; Doom rebinds it
<code>@</code>	forge dispatch. Not <code>N</code> : evil keeps <code>N</code> for search-previous
<code>SPC n p</code>	org-noter. Doom's <code>SPC n e</code> became the equations prefix
<code>SPC f r</code>	recent files. <code>SPC p r</code> was freed to be the research prefix
<code>SPC h T</code>	runs the config tests, where Doom toggles the profiler. The theme picker is <code>SPC t t</code>
<code>SPC o p</code>	Treemacs, not dirvish-side

Things that are not what they look like

<code>SPC u</code>	the universal argument. <code>C-u</code> scrolls
<code>C-p</code>	find file in project, not previous line
<code>C-s</code>	save. Search is <code>/</code> or <code>SPC s s</code>
<code>C-c C-j</code>	in a terminal, hands keys <i>to the shell</i> . <code>C-c C-e</code> hands them back to Emacs
<code>q</code>	buries the buffer in most read-only modes; it does not quit Emacs

When git refuses

The reason is almost never in the echo area. Open the process buffer with `~` in any magit buffer and read the actual git output.

When a key does nothing

<code>SPC h k</code>	shows what it is really bound to
<code>SPC h b b</code>	searches every active binding
<code>SPC ?</code>	context help for this buffer

A binding that resolves to the wrong command usually means two modules bound the same sequence; the later module in `config.el` wins.